

zerotrading-core — Accounting Schema: Fee Model Corrections

Amends: zerotrading-core-accounting.md

Scope: Corrected sections only. All other sections from the prior draft remain in force.

Source: Kalshi Fee Schedule (effective Feb 5, 2026), Kalshi Fee Rounding docs (Mar 2026), Kalshi API Fill + MarketPosition models.

What Was Wrong in the Prior Draft

The prior draft modeled Kalshi fees as:

```
fee_cents = max(0, pnl_gross_cents) × 0.07
— "7% of winning P&L, zero on losses"
```

This is incorrect in two fundamental ways:

1. **Wrong basis.** Kalshi fees are charged on **expected earnings per fill**, not on realized P&L after the trade resolves. The fee is assessed at execution time, on every fill, regardless of whether the trade eventually wins or loses.
2. **Wrong structure.** Fees apply to every taker fill. Maker fills may qualify for reduced maker fees ($0.25 \times \text{rate}$) or zero fees depending on the market. The prior model missed this entirely.
3. **Wrong placement.** The prior draft put `feeEstimatedCents` on `FillRecord`. A fill is a raw execution fact from the exchange. Fee computation is a separate accounting function that runs against fill data. The fill record should carry only what the exchange actually told us about fees, and the accounting layer does the estimation.

Kalshi Actual Fee Model

General formula (taker orders, all KXBTCD markets)

```
fee_per_fill = roundUp( $0.07 \times C \times P \times (1 - P)$ , $0.0001)
```

Where:

- C = number of contracts in this fill (fixed-point, can be fractional)
- P = fill price in dollars (e.g., $75¢ = 0.75$)
- `roundUp( $x$ , $0.0001)` = ceiling to nearest \$0.0001 (centicent)

This formula is symmetric around 50¢ and is maximized at $P = 0.50$ (2 contracts at 50¢ = \$0.035 fee).

Maker fee formula (where applicable)

```
fee_per_fill = roundUp(0.0175 × C × P × (1 - P), $0.0001)
```

$0.0175 = 0.07 \times 0.25$. Maker fee is 25% of the taker rate.

A fill is a maker fill when `Fill.is_taker = false`.

KXBTCD market rate

KXBTCD (BTC hourly) is a general market. The general fee rate of 0.07 applies. It is not an S&P500 or Nasdaq-100 market (those use 0.035).

Fee applies at fill time, not settlement

The fee is charged on every taker fill at the moment of execution. It is not deferred to settlement. It does not depend on whether the trade wins or loses. A fill at 80¢ on 10 contracts costs:

```
fee = roundUp(0.07 × 10 × 0.80 × 0.20, $0.0001)
    = roundUp(0.07 × 10 × 0.16, $0.0001)
    = roundUp(0.112, $0.0001)
    = $0.1120
    = 11.20¢
```

Whether that position wins or loses at settlement has no effect on this fee.

Fees are zero on maker fills (typically)

When the bot places an aggressive limit order that crosses the spread and immediately matches resting orders, `is_taker = true` and fees apply. When the bot posts a resting limit order that is later filled by another trader, `is_taker = false` and fees are typically zero (or at the reduced maker rate if the market is in a maker-fee program).

For the hour-harvester launch strategy, the bot places aggressive crosses to guarantee fill. Expect `is_taker = true` on essentially all fills. Maker path should not be assumed.

Rounding algorithm (per fill)

The exchange uses a multi-component rounding mechanism to keep all balances as whole cents:

1. Compute `raw_trade_fee = fee_rate × C × P × (1 - P)` [float, up to 6 decimal places]
2. `trade_fee = ceil(raw_trade_fee, $0.0001)` [rounded up to cents]
3. `revenue = C × price_dollars × sign` [revenue from this fill]
 - for a buy: `revenue = -C × P` (cost, negative)
 - for a sell: `revenue = +C × P` (proceeds, positive)
4. `balance_change = revenue - trade_fee`
5. `balance_change_floored = floor(balance_change, $0.01)` [floor to nearest cent]

```

6. rounding_fee = balance_change - balance_change_floored      [fractional remainder]
7. net_fee = trade_fee + rounding_fee                          [total fee this fill]
8. Update fee_accumulator += rounding_fee
   If fee_accumulator >= $0.01:
       rebate = $0.01
       fee_accumulator -= $0.01
       net_fee -= $0.01

```

The `fee_accumulator` is maintained **per order** across all of its fills. It ensures that splitting one order into N partial fills produces the same total fee as a single equivalent fill.

The exchange returns the actual `fee_cost` per fill in `Fill.fee_cost` (dollar string, up to 6 decimal places). This is the net fee after rounding and rebates. Do not recompute it — read it.

Settlement fees

For standard yes/no binary markets (which KXBTCD is), settlement fees are zero for most positions. A sub-cent settlement fee (`MiscFeeAmt`) may arise only from scalar settlement or fractional contract positions. For integer-contract KXBTCD positions, treat expected settlement fee as zero and flag any non-zero `MiscFeeAmt` in the reconciliation event.

What the exchange returns

On each fill (`GET /portfolio/fills`):

- `Fill.fee_cost` — dollar string, the actual net fee for this fill (after rounding + rebate). This is authoritative.
- `Fill.is_taker` — bool. Whether fees applied at taker rate.
- `Fill.yes_price_dollars / Fill.no_price_dollars` — execution price.
- `Fill.count_fp` — fill size (fixed-point string, 2 decimals).

On the position (`GET /portfolio/positions`):

- `MarketPosition.fees_paid_dollars` — cumulative fees paid across all fills for this market. Dollar string, up to 6 decimal places.
- `MarketPosition.realized_pnl_dollars` — exchange-computed realized P&L. This is **net of fees** as computed by the exchange. Dollar string.

The relationship is:

```

realized_pnl_dollars (exchange) = gross_proceeds - entry_cost - fees_paid_dollars

```

All of these are authoritative exchange figures. When available, they supersede any bot estimation.

Corrected: Accounting Principles

Replace principle 3 from the prior draft with the following:

3. Fee accounting is per-fill, not per-settlement. Kalshi fees are charged at taker fill execution time on every fill. The fee formula is $\text{roundUp}(0.07 \times C \times P \times (1-P), \$0.0001)$ where P is the fill price in dollars and C is the contract count. The fee does not depend on whether the position eventually wins or loses. The exchange returns the actual fee in `Fill.fee_cost`; this field is authoritative. Bot estimates are used only when the fill has not yet been confirmed by the exchange. Maker fills (`is_taker = false`) are typically zero-fee and must be tracked separately.

Add a new principle:

3a. Exchange-confirmed values supersede bot estimates at every boundary. When a `Fill` is confirmed by the exchange: use `fee_cost` (not the formula). When a `MarketPosition` is confirmed settled: use `realized_pnl_dollars` and `fees_paid_dollars` (not bot-computed gross minus fee). Bot estimates exist only to bridge the gap before exchange confirmation arrives. They are never promoted to confirmed status — they are replaced.

Corrected: FillRecord

The prior draft placed `feeEstimatedCents` and `feeActualCents` on `FillRecord`. This conflates a raw execution record with an accounting computation. A fill is a fact. Fee estimation is a function applied to that fact by the accounting layer.

```
interface FillRecord {
  fillId: FillId;           // exchange-assigned fill UUID (Fill.fill_id)
  orderId: OrderId;         // parent order (Fill.order_id)
  positionId: PositionId;

  ticker: string;
  action: OrderAction;      // "buy" | "sell"
  side: OrderSide;          // "yes" | "no"

  // Execution fact – exactly what the exchange reported
  countFp: string;          // Fill.count_fp – fixed-point string, e.g. "10.00"
  count: number;            // parsed numeric value of countFp
  fillPriceDollars: string;  // Fill.yes_price_dollars or no_price_dollars – raw
  fillPriceCents: number;    // Math.round(parseFloat(fillPriceDollars) * 100) –
  isTaker: boolean;         // Fill.is_taker – drives fee rate in accounting layer

  // Exchange-reported fee – present when source = "exchange_confirmed"
  // Fill.fee_cost is a dollar string (e.g., "0.112000")
  // Stored as-is for auditability; converted to centicents for accounting
  feeCostDollars: string | null; // null until exchange confirms
  feeCostCenticents: number | null; // Math.round(parseFloat(feeCostDollars) * 10000)
                                   // null until exchange confirms

  filledAt: number;         // epoch ms
```

```

    source: "exchange_confirmed" | "bot_inferred";
    // "bot_inferred" if we determined the fill from position delta before /fills cor
    // Must be upgraded to "exchange_confirmed" when the exchange fill arrives.
    // feeCostDollars / feeCostCenticents remain null until source = "exchange_confirm
  }

```

Why feeCostCenticents (not cents)?

Kalshi's fee formula produces values like \$0.0120, which is 1.20¢ — not a whole cent. Storing as centicents (integer units of \$0.0001) preserves the precision the exchange computes at. All aggregations and ledger entries convert to whole cents at the ledger boundary using ceiling rounding, matching exchange behavior.

Corrected: Fee Estimation Function

The accounting layer uses this function when a fill exists but exchange confirmation has not yet arrived:

```

// Fee rate by fill role
const FEE_RATE_TAKER = 0.07;
const FEE_RATE_MAKER = 0.0175; // 25% of taker; used when isTaker = false

function estimateFeeForFill(fill: FillRecord): number {
  // Returns fee in centicents (integer, $0.0001 units)
  // Uses ceiling rounding to match exchange behavior.
  const P = fill.fillPriceCents / 100; // price in dollars
  const C = fill.count; // contract count
  const rate = fill.isTaker ? FEE_RATE_TAKER : FEE_RATE_MAKER;

  // raw fee in dollars, up to 6 decimal places
  const rawFeeDollars = rate * C * P * (1 - P);

  // Round up to nearest $0.0001 (centicent)
  const feeCenticents = Math.ceil(rawFeeDollars * 10000);

  // Note: this estimate does NOT account for the rounding_fee component or
  // the fee_accumulator rebate mechanism. It approximates the trade_fee
  // component only. The difference is always < $0.01 per fill.
  // Reconciliation corrects the delta when exchange fill is confirmed.
  return feeCenticents;
}

// For the ledger, convert centicents to whole cents using ceiling:
function centicentsToCeilCents(centicents: number): number {
  return Math.ceil(centicents / 100);
}

```

This function is only called by the accounting layer when writing a `fee_estimated` `LedgerEntry`. It is never called directly by the engine's trading logic.

Corrected: LedgerEntry Fee Types

Replace the fee-related LedgerEntryType values with:

```
type LedgerEntryType =
  | "entry_cost"           // capital deployed at entry fill: debit
  | "exit_proceeds"        // capital returned at exit fill: credit
  | "fee_taker_estimated"  // bot-estimated taker fee at fill time (before exchange fee)
  | "fee_maker_estimated"  // bot-estimated maker fee (typically 0 or 0.25× take)
  | "fee_confirmed"        // exchange-confirmed fee from Fill.fee_cost: replace
  | "fee_correction"       // reconciliation delta: confirmed - estimated (signed)
  | "settlement_fee"       // MiscFeeAmt from settlement (expected 0 for integer-count)
  | "pnl_realized_bot"     // bot-computed realized P&L when exit fill confirmed
  | "pnl_realized_reconciled" // exchange-confirmed realized P&L (supersedes bot e
  | "pnl_correction"       // reconciliation delta between bot and exchange P&L
  | "mtm_unrealized";      // mark-to-market unrealized snapshot (informational,
```

Fee ledger entry lifecycle for a single taker fill

At fill confirmation (bot has confirmed fill exists, exchange fee not yet fetched):

```
Write: LedgerEntry(fee_taker_estimated)
  amountCents    = -centicentsToCeilCents(estimateFeeForFill(fill)) [negative = cost]
  feesAmountCents = same
  fillId         = fill.fillId
  source         = "bot_estimated"
  memo           = "Taker fee estimate: roundUp(0.07 × {C} × {P} × {1-P}) = {result}'
```

When exchange confirms Fill.fee_cost:

```
Write: LedgerEntry(fee_confirmed)
  amountCents    = -Math.ceil(parseFloat(fill.feeCostDollars) * 100) [ceiling to cents]
  feesAmountCents = same
  fillId         = fill.fillId
  source         = "exchange_confirmed"
  memo           = "Exchange-confirmed fee from Fill.fee_cost: ${fill.feeCostDollars}"
```

```
Write: LedgerEntry(fee_correction) IF confirmed ≠ estimated
  amountCents    = confirmed_cents - estimated_cents [signed delta, may be 0]
  originalLedgerEntryId = the fee_taker_estimated entry id
  correctionDeltaCents = same as amountCents
  source         = "reconciliation_correction"
  memo           = "Fee correction: estimated {estimated_cents}¢ → confirmed {confirmed_cents}¢"
```

Fee ledger entry for settlement

For standard binary KXBTCD settlement:

```
Expected: no settlement fee entry (MiscFeeAmt = 0 for integer-count binary positions)
If MiscFeeAmt ≠ 0:
```

```

Write: LedgerEntry(settlement_fee)
    amountCents = -Math.ceil(MiscFeeAmt × 100)
    source      = "exchange_confirmed"
    memo        = "Settlement rounding fee from MiscFeeAmt"

```

Corrected: PositionRecord Fee Fields

The realizedPnl sub-object on PositionRecord needs updated fee fields:

```

// Replace the realizedPnl block in PositionRecord with:
realizedPnl: {
    // Gross: sum of (exitFillPrice - entryAvgPrice) × fillCount across all exit fills
    // Denominated in whole cents.
    grossCents: number;

    // Fees: tracked in centicents for precision, converted to whole cents for P&L
    feesEstimatedCenticents: number;    // sum of estimateFeeForFill() across all fill
    feesConfirmedCenticents: number | null; // sum of Fill.fee_cost × 10000 when excha
                                           // null until all fills confirmed

    // Net: grossCents minus fee in whole cents (ceiling rounding on fees to match exc
    netEstimatedCents: number;           // grossCents - centicentsToCeilCents(feesEsti
    netConfirmedCents: number | null;    // grossCents - centicentsToCeilCents(feesCon
                                           // null until all fills confirmed

    source: "bot_estimated" | "exchange_confirmed";
    // "exchange_confirmed" only after reconciliation reads realized_pnl_dollars + fee
    // from MarketPosition and the values are consistent with fill records.

    confirmedAt: number | null;
} | null;

```

Clarification on entry fees:

Entry fees (paid when buying in) reduce net P&L. Exit fees (paid when selling out early) also reduce net P&L. Settlement has no fee for standard binary positions. The feesEstimatedCenticents field accumulates fees from **all fills**: entry and exit. Entry fees are booked to LedgerEntry at entry fill time, not deferred.

Corrected: Reconciliation Rules — Fee and Settlement Sections

Rule 1A: Fill fee confirmation (replaces prior Rule 1)

Trigger: Any FillRecord with source = "exchange_confirmed" and feeCostDollars = null.

Action: Query GET /portfolio/fills?order_id={orderId}. Find the matching fill by fill_id. Read fee_cost.

On success: Set fill.feeCostDollars, compute fill.feeCostCenticents. Compare to estimated.

- If delta = 0: emit ReconciliationEvent(no_discrepancy), field=fee.

- If $\text{delta} \neq 0$: emit `ReconciliationEvent(fee_correction)`. Write `fee_confirmed` and `fee_correction` `LedgerEntries`.
Idempotency key: `{positionId}:fee_confirmed:{fillId}:{feeCostDollars}`.

Rule 2A: Settlement confirmation (replaces prior Rule 2)

Trigger: Position in `closed_estimated`.

Action: Query GET `/portfolio/positions?settlement_status=settled`. Find position by ticker.

Exchange provides: `realized_pnl_dollars` (net, after fees) and `fees_paid_dollars`.

Reconciliation logic:

```
exchange_net_cents = Math.round(parseFloat(realized_pnl_dollars) * 100)
exchange_fees_centicents = Math.round(parseFloat(fees_paid_dollars) * 10000)
bot_net_cents = position.realizedPnl.netEstimatedCents

delta_cents = exchange_net_cents - bot_net_cents
```

- If $\text{delta} = 0$: `ReconciliationEvent(no_discrepancy)`. Position $\rightarrow$ `closed_confirmed`. Set `realizedPnl.source = "exchange_confirmed"`.
- If $\text{delta} \neq 0$: `ReconciliationEvent(pnl_correction)` with `deltaCents = delta_cents`. Write `pnl_correction` `LedgerEntry`. Set `realizedPnl.netConfirmedCents = exchange_net_cents`, `feesConfirmedCenticents = exchange_fees_centicents`, `source = "exchange_confirmed"`. Position $\rightarrow$ `closed_confirmed`.

Important: Use `realized_pnl_dollars` from the exchange as the settlement truth. Do not re-derive it from the payout formula. The formula ($\text{payout} = (100 - \text{entryPrice}) \times \text{count}$ for a win) is a fallback used only when the exchange endpoint is unavailable and the market close time has passed. When the exchange value is available, it takes precedence unconditionally.

Idempotency key: `{positionId}:settlement_confirmed:{exchange_net_cents}:{exchange_fees_centicents}`.

Corrected: Invariants

Replace the fee-related invariants from the prior draft:

INV-3a (replaces prior fee invariant):

For every confirmed exit fill:

`LedgerEntry(fee_taker_estimated OR fee_confirmed)` MUST exist with `fillId = fill`.
`amountCents < 0` (fees are always a cost, never zero on taker fills)
EXCEPT when `fill.isTaker = false` (maker fill may have zero fee)

INV-3b (new):

`LedgerEntry(fee_confirmed).amountCents` is derived exclusively from `Fill.feeCostDc`.
It is NEVER derived from the fee formula.
The fee formula is used ONLY to produce `fee_taker_estimated` entries.

INV-3c (new):

```
position.realizedPnl.feesConfirmedCenticents =  
    sum(fill.feeCostCenticents) for all fills (entry + exit) on this position  
This equality must hold once source = "exchange_confirmed".
```

INV-3d (new):

```
position.realizedPnl.netConfirmedCents =  
    Math.round(parseFloat(marketPosition.realized_pnl_dollars) * 100)  
i.e., net confirmed P&L comes from the exchange's realized_pnl_dollars field directly  
not from gross - fees computed locally.
```

INV-4 (updated):

```
For entries of type pnl_realized_bot:  
    amountCents = grossAmountCents + feesAmountCents  
    where feesAmountCents = -centicentsToCeilCents(feesEstimatedCenticents)
```

INV-5 (updated):

```
For every position in closed_confirmed:  
    sum(LedgerEntry.amountCents WHERE positionId = X AND type NOT IN ('mtm_unrealized', 'pnl_realized_bot')  
    ≈ position.realizedPnl.netConfirmedCents  
    within 1 cent tolerance (from ceiling rounding of centicent fees)
```

Remove prior INV-4 ("LedgerEntry.amountCents = grossAmountCents + feesAmountCents") as stated — this is now captured by the more precise INV-4 updated above.

Corrected: BreakerAccounting Fee Treatment

Replace the fee comment in BreakerAccounting:

```
interface BreakerAccounting {  
    // dailyRealizedNetCents sources:  
    //   pnl_realized_bot entries (bot-estimated, used until exchange confirms)  
    //   pnl_realized_reconciled entries (exchange-confirmed, replaces bot estimate)  
    //   pnl_correction entries (reconciliation delta)  
    //   fee_confirmed entries (when they differ from fee_taker_estimated)  
    //   fee_correction entries  
    // Does NOT include: mtm_unrealized, fee_taker_estimated (superseded by fee_confirmed)  
    // When both fee_taker_estimated AND fee_correction exist for the same fill,  
    // only the corrected total counts (the correction entry carries the delta, both sources are ignored)  
    dailyRealizedNetCents: number;  
    hourlyRealizedNetCents: number;  
    consecutiveLosses: number;  
    openRiskCents: number;  
    effectiveDailyExposureCents: number;  
    computedAt: number;  
}
```

Summary: What Changed

Prior model	Corrected model
Fee = 7% of winning P&L, zero on losses	$\text{Fee} = \text{roundUp}(0.07 \times C \times P \times (1-P), \$0.0001)$ per fill, at execution time, regardless of outcome
Fee applies only at settlement	Fee applies at every taker fill execution
feeEstimatedCents and feeActualCents on FillRecord	FillRecord carries only raw exchange data (feeCostDollars, feeCostCenticents); estimation is an accounting-layer function
Fee precision: whole cents	Fee precision: centicents (\$0.0001) internally; converted to whole cents at ledger boundary via ceiling
Single fee field	Four fee LedgerEntry types: fee_taker_estimated, fee_maker_estimated, fee_confirmed, fee_correction
Maker vs taker not modeled	is_taker flag on FillRecord drives fee rate; maker fills treated as zero-fee by default
Settlement P&L derived from formula	Settlement P&L sourced from realized_pnl_dollars on MarketPosition; formula is fallback only
Fee accumulator / rounding not modeled	Rounding and accumulator acknowledged; bot estimate approximates trade_fee component only; delta corrected by reconciliation
Entry fees not mentioned	Entry fill fees booked as fee_taker_estimated LedgerEntry at entry fill time